

UNIVERSIDADE FEDERAL DO AMAZONAS
INSTITUTO DE COMPUTAÇÃO – ICOMP
ENGENHARIA DA COMPUTAÇÃO
INTELIGÊNCIA ARTIFICIAL

BEATRIZ AUGUSTA COELHO BEZERRA
DANIEL RAMOS MAIA
VICTOR LIMA FRAZÃO

1º Trabalho Prático
Representação do Conhecimento para Gerar Planejador que Empilha
Blocos de Diferentes Dimensões

Manaus - AM

2026

BEATRIZ AUGUSTA COELHO BEZERRA
DANIEL RAMOS MAIA
VICTOR LIMA FRAZÃO

1º Trabalho Prático

Representação do Conhecimento para Gerar Planejador que Empilha Blocos de Diferentes Dimensões

Trabalho prático sobre representação do conhecimento para gerar planejador que empilha blocos de diferentes dimensões como requisito para obtenção de nota na disciplina de Inteligência Artificial ministrado pelo professor Edjard Mota da Universidade Federal do Amazonas.

MANAUS - AM

2026

1. PRIMEIRA QUESTÃO – DESCRIÇÃO DO PROBLEMA EM LINGUAGEM NATURAL

O problema consiste no empilhamento de blocos retangulares que possuem diferentes larguras, mas todos com a mesma altura unitária (altura = 1). Os blocos são identificados pelas letras a,b,c e d, com as seguintes larguras:

Bloco	Largura	Posições Ocupadas
a	1	1
b	1	1
c	2	2
d	3	3

Os blocos são posicionados sobre uma superfície plana (a mesa) com 7 posições horizontais numeradas de 0 a 6. Cada bloco ocupa uma sequência contínua de posições horizontais correspondente á sua largura, e pode ser empilhado sobre outros blocos, desde que respeite as regras de estabilidade física.

O estado inicial (S0) das Situações 1 e 3 é:

- a: começa em 3, termina em 4;
- b: começa em 5, termina em 6;
- c: começa em 0 termina em 2
- d: começa em 3 e termina em 6 (está em cima de 'a' e 'b')

O estado inicial (S0) Situação 2:

- a: começa em 0, termina em 1; está em cima de c
- b: começa em 1, termina em 2; está em cima de c
- c: começa em 0 termina em 2
- d: começa em 3 e termina em 6

O objetivo em cada situação é encontrar uma sequência de movimentos que move o bloco B da posição P_i para a posição P_j , que transforme o estado inicial em uma configuração final desejada. As regras que o mundo deve respeitar são:

- Um bloco só pode ser movido se não houver nenhum outro bloco em cima dele;

- Um bloco só pode ser colocado em uma posição se todas as posições que ele ocupa estiverem livres;
- Um bloco só pode ser empilhado sobre outro se houver suporte suficiente abaixo dele
- Os blocos não podem ultrapassar os limites da mesa;
- Um bloco mais largo não pode ficar em cima de um mais estreito sem suporte adequado.

2. SEGUNDA QUESTÃO – DEFINIÇÃO DOS CONCEITOS

1. block (B, W) - cadastro dos blocos: Define que o bloco B tem largura W. É o cadastro do mundo.

block(a, 1).

block(b, 1).

block(c, 2).

block(d, 3).

2. on(B, X) - sobreposição: Define o bloco B está em cima de X, onde X pode ser outro bloco ou o chão (floor).

on(d, a).

on(d, b).

on(a, floor).

on(c, floor).

on(d, floor).

3. position(B, X1, X2) - posição horizontal: Define que o bloco B ocupa da posição X1 até X2 na mesa.

position(a, 3, 4).

position(b, 5, 6).

position(c, 0, 2).

position(d, 3, 6).

4. clear(B) - bloco livre: Um bloco está livre quando não tem nenhum outro bloco em cima dele. Só blocos livres podem ser movidos.

clear(c).

clear(d).

5. overlap(B1, B2) - dois blocos se sobrepõem quando ocupam pelo menos uma posição.

6. center (B, C) - define o centro geométrico do bloco B, calculado como:
$$C = (X1 + X2) / 2$$

7. stable(B) - um bloco é considerado estável quando seu centro geométrico está de dentro da base de suporte.

3. TERCEIRA QUESTÃO – AÇÕES DO SISTEMA

A ação principal do planejador é mover um bloco de uma posição para outra, definida como move(B, Pi, Pj), onde B é o bloco a ser movido, Pi é a posição de origem e Pj é a posição de destino.

Para que a ação seja executada, as seguintes pré-condições precisam ser verdadeiras:

can(move(B, Pi, Pj), [clear(B), on(B, Pi), position(B, Xi, Xf), free_space(Yj, Yf), stable(B, Pj)]).

Onde:

- clear(B) garante que não há nenhum bloco em cima de B
- on(B, Pi) garante que B está na posição de origem Pi
- position(B, Xi, Xf) confirma a posição horizontal atual de B
- free_space(Yj, Yf) garante que todas as posições de destino estão

livres

- stable(B, Pj) garante que B ficará estável na posição de destino

Os efeitos positivos da ação, ou seja, o que passa a ser verdade depois do movimento, são:

adds(move(B, Pi, Pj), [on(B, Pj), position(B, Yj, Yf), clear(Pi)]).

Os efeitos negativos da ação, ou seja, o que deixa de ser verdade depois do movimento, são:

deletes(move(B, Pi, Pj), [on(B, Pi), position(B, Xi, Xf), clear(Pj)]).

A regra de estabilidade garante que um bloco só pode ser empilhado sobre outro se o bloco de cima for menor ou igual ao bloco de baixo em largura:

stable(B1, B2) :- block(B1, W1), block(B2, W2), $W1 \leq W2$.

Para o ponto extra de equilíbrio por centro geométrico, o centro de um bloco é calculado como:

center(B, C) :- position(B, X1, X2), C is $(X1 + X2) / 2$.

Um bloco B1 está em equilíbrio sobre B2 quando o centro de B1 está dentro dos limites horizontais de B2:

balanced(B1, B2) :- center(B1, C1), position(B2, X1, X2), C1 >= X1, C1 =< X2.

4. QUARTA QUESTÃO – RESTRIÇÕES EM LINGUAGEM NATURAL

As restrições do mundo dos blocos com dimensões diferentes são:

1. Um bloco só pode ser movido se estiver livre, ou seja, não pode haver nenhum outro bloco em cima dele.

2. A posição de destino deve estar dentro dos limites da mesa O bloco não pode ultrapassar as posições 0 a 6.

3. Todas as posições que o bloco ocupará no destino devem estar livres Como d tem largura 3, as 3 posições de destino precisam estar livres.

4. O bloco de cima não pode ser mais largo que o bloco de baixo Por exemplo, d (largura 3) não pode ficar em cima de c (largura 2).

5. Um bloco não pode ser movido para o lugar onde já está A posição de origem e destino devem ser diferentes.

6. Dois blocos não podem ocupar o mesmo lugar Nenhuma posição pode ser ocupada por dois blocos ao mesmo tempo.

Agora em Prolog, usando a estrutura do Bratko:

% Restrição 1 - bloco deve estar livre

can(move(B, Pi, Pj), _) :-

\+ clear(B), !, fail.

% Restrição 2 - destino dentro dos limites

can(move(B, Pi, Pj), _) :-

block(B, W),

Pj + W > 6, !, fail.

% Restrição 3 - posições de destino livres

free_space(Pj, W) :-

End is Pj + W,

base_limits(Min, Max),

Pj >= Min,

End <= Max.

% Restrição 4 - estabilidade

stable(B1, B2) :-

block(B1, W1),

block(B2, W2),

W1 <= W2.

% Restrição 5 - origem diferente do destino

can(move(B, Pi, Pj), _) :-

Pi <= Pj, !, fail.

% Restrição 6 - impossibilidade lógica

impossible(on(X, X), _).

impossible(on(X, Y), Goals) :-

member(clear(Y), Goals)

;

member(on(X, Y1), Goals), Y1 <= Y

;

member(on(X1, Y), Goals), X1 <= X.

impossible(clear(X), Goals) :-

member(on(_, X), Goals).

4.1 CÓDIGO DO LIVRO DO BRATKO

O planejador utilizado é baseado em regressão de metas (goal regression) com busca em profundidade iterativa, conforme descrito no Capítulo 17 do livro de Bratko. O predicado principal é `plan(State, Goals, Plan)`, que gera uma sequência de ações `Plan` que transforma o estado `State` no estado objetivo `Goals`.

% Caso base: plano vazio quando todos os objetivos já estão satisfeitos

plan(State, Goals, []) :-

satisfied(State, Goals).

% Passo recursivo: encontra uma ação que resolve um objetivo

plan(State, Goals, Plan) :-

```
append(PrePlan, [Action], Plan),
select(State, Goals, Goal),
achieves(Action, Goal),
can(Action, Condition),
preserves(Action, Goals),
regress(Goals, Action, RegressedGoals),
plan(State, RegressedGoals, PrePlan).
```

% Verifica se todos os objetivos estão satisfeitos no estado atual

satisfied(State, Goals) :-

```
delete_all(Goals, State, []).
```

% Seleciona um objetivo da lista de objetivos

select(_, Goals, Goal) :-

```
member(Goal, Goals).
```

% Verifica se uma ação alcança um objetivo

achieves(Action, Goal) :-

```
adds(Action, Goals),
```

```
member(Goal, Goals).
```

% Verifica se uma ação preserva os outros objetivos

preserves(Action, Goals) :-

```
deletes(Action, Relations),
```

```
\+ (member(Goal, Relations),
```

```
member(Goal, Goals)).
```

% Regressão dos objetivos através da ação

regress(Goals, Action, RegressedGoals) :-

```
adds(Action, NewRelations),
```

```
delete_all(Goals, NewRelations, RestGoals),
```

```
can(Action, Condition),
```

```
addnew(Condition, RestGoals, RegressedGoals).
```



```
% Adiciona novos objetivos verificando incompatibilidades
```

```
addnew([], L, L).
```

```
addnew([Goal|_], Goals, _) :-
```

```
    impossible(Goal, Goals), !,
```

```
    fail.
```

```
addnew([X|L1], L2, L3) :-
```

```
    member(X, L2), !,
```

```
    addnew(L1, L2, L3).
```

```
addnew([X|L1], L2, [X|L3]) :-
```

```
    addnew(L1, L2, L3).
```

```
% Remove elementos de uma lista
```

```
delete_all([], _, []).
```

```
delete_all([X|L1], L2, Diff) :-
```

```
    member(X, L2), !,
```

```
    delete_all(L1, L2, Diff).
```

```
delete_all([X|L1], L2, [X|Diff]) :-
```

```
    delete_all(L1, L2, Diff).
```

4.2 IMPLEMENTAÇÃO DO PLANEJADOR PARA BLOCOS DE DIMENSÕES DIFERENTES

O planejador foi implementado adaptando o código de goal regression do Bratko para suportar blocos de larguras diferentes. A principal mudança em relação ao mundo de blocos simples é que agora cada bloco ocupa múltiplas posições horizontais, e as regras de estabilidade precisam verificar se o bloco de cima cabe sobre o bloco de baixo.

4.2.1 DEFINIÇÃO DO MUNDO

```
% Largura de cada bloco
```

```
block(a, 1).
```

```
block(b, 1).
```

```
block(c, 2).
```

```
block(d, 3).
```

```
% Limites da mesa
```

```
base_limits(0, 6).
```

```
% Objetos do mundo (blocos e chão)
```

```
object(X) :- block(X, _).
```

```
object(floor).
```

4.2.2 ESTADOS INICIAIS

```
% Estado inicial S0 - Situações 1 e 3
```

```
state_s0_sit1([  
    on(a, floor), position(a, 3, 4),  
    on(b, floor), position(b, 5, 6),  
    on(c, floor), position(c, 0, 2),  
    on(d, a),    position(d, 3, 6),  
    clear(c),  
    clear(d),  
    clear(b)  
]).
```

```
% Estado inicial S0 - Situação 2
```

```
state_s0_sit2([  
    on(a, c),    position(a, 0, 1),  
    on(b, c),    position(b, 1, 2),  
    on(c, floor), position(c, 0, 2),  
    on(d, floor), position(d, 3, 6),  
    clear(a),  
    clear(b),  
    clear(d)  
]).
```

```
%
```

4.2.3 REGRAS DE ESTABILIDADES

```
% Centro geométrico de um bloco (ponto extra)
```

```
center(B, C) :-
```

```
    block(B, W),
```

```
position(B, X1, _),  
C is X1 + W / 2.
```

% Um bloco é estável sobre outro se for menor ou igual em largura

```
stable(B1, B2) :-  
    block(B1, W1),  
    block(B2, W2),  
    W1 <= W2.
```

% Um bloco é estável sobre dois blocos pelo centro geométrico (ponto extra)

```
stable_center(B1, B2, B3) :-  
    block(B1, W1),  
    center(B1, C1),  
    position(B2, X1, _),  
    position(B3, _, X2),  
    C1 >= X1,  
    C1 <= X2,  
    W1 <= (X2 - X1).
```

4.2.4 AÇÕES

% Mover bloco para o chão

```
can(move(B, Pi, floor), [clear(B), on(B, Pi), position(B, Xi, Xf)]) :-  
    block(B, W),  
    base_limits(Min, Max),  
    Xi >= Min,  
    Xf <= Max,  
    Pi \== floor.
```

% Mover bloco para cima de outro bloco

```
can(move(B1, Pi, B2), [clear(B1), clear(B2), on(B1, Pi), position(B2, _, _)])
```

:-

```
    block(B1, _),  
    block(B2, _),
```

```
B1 \== B2,  
Pi \== B2,  
stable(B1, B2).
```

% O que a ação adiciona ao estado
adds(move(B, Pi, Pj), [on(B, Pj), clear(Pi)]).

% O que a ação remove do estado
deletes(move(B, Pi, Pj), [on(B, Pi), clear(Pj)]).

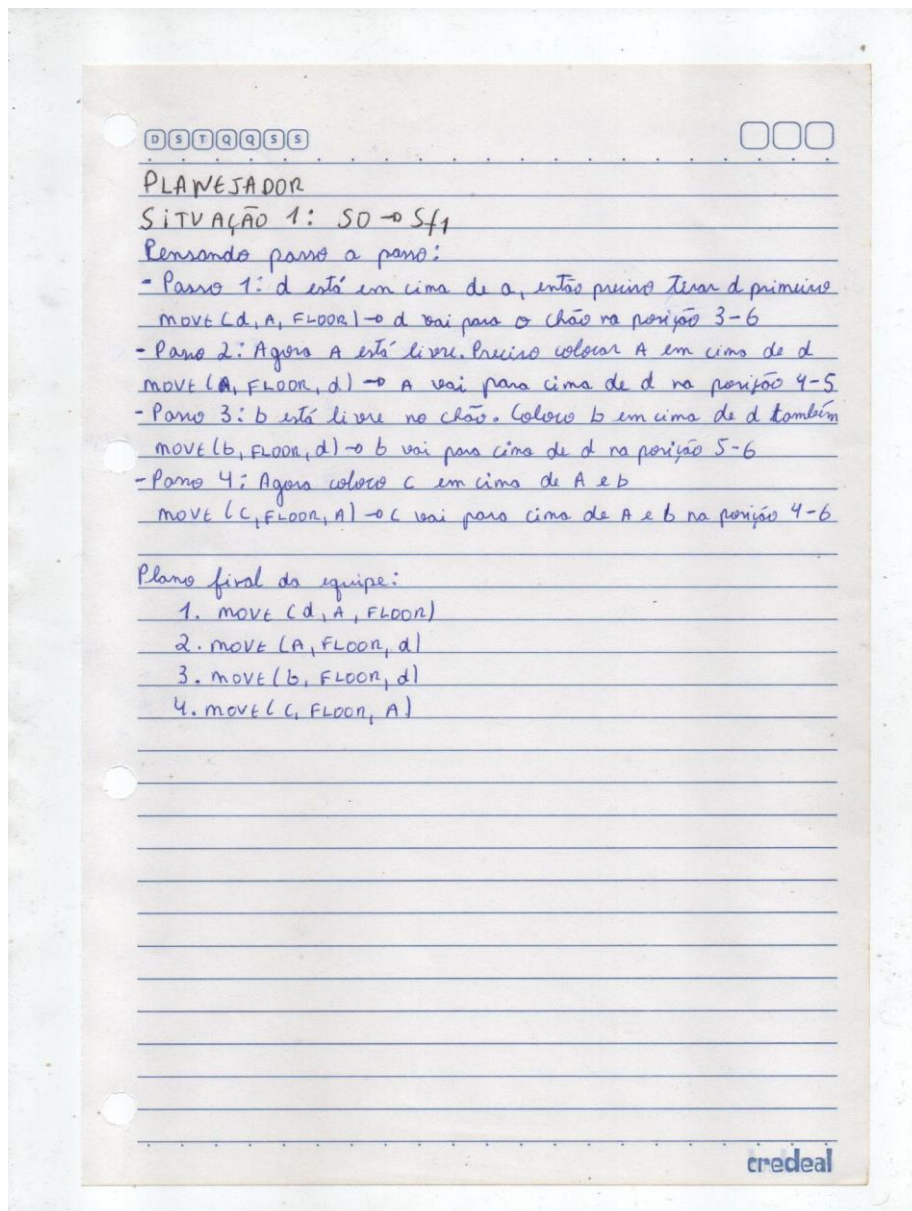
4.2.5 IMPOSSIBILIDADES

```
impossible(on(X, X), _).  
impossible(on(X, Y), Goals) :-  
    member(clear(Y), Goals)  
    ;  
    member(on(X, Y1), Goals), Y1 \== Y  
    ;  
    member(on(X1, Y), Goals), X1 \== X.  
impossible(clear(X), Goals) :-  
    member(on(_, X), Goals).
```

4.2.6 PLANEJADOR (BRATKO)

Situação 1: Planos Manuais — Situação 1: S0 → Sf1

Plano da equipe



Plano da IA

A IA pensa de trás para frente (goal regression):

Objetivo final: $\text{on}(c, a)$, $\text{on}(a, d)$, $\text{on}(b, d)$, $\text{on}(d, \text{floor})$

Regredindo: Para $\text{on}(c, a)$ ser verdade, preciso de $\text{clear}(c)$ e $\text{clear}(a)$. c já está livre. Para a estar livre, d precisa sair de cima de a .

Regredindo: Para $\text{on}(d, \text{floor})$, preciso mover d do lugar atual. Mas d está em cima de a , então d está livre — posso mover.

Sequência encontrada:

1. $\text{move}(d, a, \text{floor})$ → tira d de cima de a , d vai para posição 3-6
2. $\text{move}(a, \text{floor}, d)$ → coloca a em cima de d na posição 4-5
3. $\text{move}(b, \text{floor}, d)$ → coloca b em cima de d na posição 5-6

4. `move(c, floor, a)` → coloca c em cima de a e b na posição 4-6

Comparação Situação 1

	Plano do Aluno	Plano da IA
Número de passos	4	4
Primeiro movimento	<code>move(d, a, floor)</code>	<code>move(d, a, floor)</code>
Estratégia	frente para trás	trás para frente
Resultado	igual	igual

Situação 2: Planos Manuais — Situação 1: S0 → S5

Plano da equipe

DSTQQSS

000

SITUAÇÃO 2: $S_0 \rightarrow S_5$

Passo 1: A e b estão em cima de C. Precisamos tirá-lo para mover C. Tiro A primeiro

$MOVE(A, C, FLOOR) \rightarrow A$ vai para o chão

Passo 2: Tiro b de cima de C

$MOVE(b, C, FLOOR) \rightarrow b$ vai para o chão

Passo 3: Agora C está livre. Coloco C em cima de d

$MOVE(C, FLOOR, d) \rightarrow C$ vai para cima de d na posição 4-6

Passo 4: Coloco A em cima de C

$MOVE(A, FLOOR, C) \rightarrow A$ vai para cima de C na posição 4-5

Passo 5: Coloco b em cima de C

$MOVE(b, FLOOR, C) \rightarrow b$ vai para cima de C na posição 5-6

Novo plano:

1. $MOVE(A, C, FLOOR)$

2. $MOVE(b, C, FLOOR)$

3. $MOVE(C, FLOOR, d)$

4. $MOVE(A, FLOOR, C)$

5. $MOVE(b, FLOOR, C)$

credeal

Plano da IA

Objetivo final: $on(b, c), on(a, c), on(c, d), on(d, floor)$

Regredindo: Para $on(c, d)$, preciso de $clear(c)$. Mas a e b estão sobre c.

Logo preciso mover a e b primeiro.

Regredindo: Para $on(a, c)$ no final, preciso de $clear(a)$ e $clear(c)$. c só fica livre depois de mover a e b.

Sequência encontrada:

1. $move(a, c, floor) \rightarrow$ tira a de cima de c

2. move(b, c, floor) → tira b de cima de c
3. move(c, floor, d) → coloca c em cima de d na posição 4-6
4. move(a, floor, c) → coloca a em cima de c na posição 4-5
5. move(b, floor, c) → coloca b em cima de c na posição 5-6

Comparação Situação 2

	Plano do Aluno	Plano da IA
Número de passos	5	5
Estratégia	frente para trás	trás para frente
Resultado	igual	igual

Situação 3: Planos Manuais — Situação 1: S0 → S7

Plano da equipe

D S T Q Q S S

□ □ □

SITUAÇÃO 3: $S_0 \rightarrow S_7$

Passo 1: d está em cima de A. Precisamos tirar d primeiro
 $move(d, A, FLOOR) \rightarrow d$ vai para o chão posição 3-6

Passo 2: Agora A está livre. Coloco A em cima de C
 $move(A, FLOOR, C) \rightarrow A$ vai para cima de C na posição 0-1

Passo 3: b está livre. Coloco b em cima de C
 $move(b, FLOOR, C) \rightarrow b$ vai para cima de C na posição 1-2

Passo 4: Agora preciso mover C com A e b em cima, mas não posso. Preciso tirar A e b primeiro. Então refaço:

Passo 2 (corrigido): Movo C para a posição correta primeiro
 $move(C, FLOOR, FLOOR) \rightarrow C$ vai para posição 0-2

Passo 3 (corrigido): Coloco A em cima de C
 $move(A, FLOOR, C) \rightarrow A$ em cima de C posição 0-1

Passo 4 (corrigido): Coloco b em cima de C
 $move(b, FLOOR, C) \rightarrow b$ em cima de C na posição 1-2

Novo plano:

1. $move(d, A, FLOOR)$
2. $move(A, FLOOR, C)$
3. $move(b, FLOOR, C)$

credeal

Plano da IA

Objetivo final: $on(b, c), on(a, c), on(c, floor), on(d, floor)$

Regredindo: Para $on(a, c)$, preciso de $clear(a)$. Mas d está sobre a. Logo movo d primeiro.

Regredindo: Para $on(d, floor)$, d já está livre — posso mover diretamente.

Sequência encontrada:

1. move(d, a, floor) → tira d de cima de a, d vai para posição 3-6
2. move(a, floor, c) → coloca a em cima de c na posição 0-1
3. move(b, floor, c) → coloca b em cima de c na posição 1-2

Comparação Situação 3

	Plano do Aluno	Plano da IA
Número de passos	3	3
Estratégia	frente para trás	trás para frente
Resultado	igual	igual

5. QUESTÃO 5 – COMPARAÇÃO ENTRE PLANOS MANUAIS E PLANOS GERADOS PELO CÓDIGO

5.1 CÓDIGO

5.1.1 DECLARAÇÕES

- :- discontiguous can/2.
- :- discontiguous adds/2.
- :- discontiguous deletes/2.

5.1.2 DEFINIÇÃO DO MUNDO

```
block(a, 1).  
block(b, 1).  
block(c, 2).  
block(d, 3).  
object(X) :- block(X, _).  
object(floor).
```

5.1.3 ESTABILIDADE

```
stable(B1, B2) :-  
    block(B1, W1),  
    block(B2, W2),  
    W1 =< W2.
```

5.1.4 AÇÕES

can(move(B, Pi, floor), [clear(B), on(B, Pi)]) :-

block(B, _),

block(Pi, _),

Pi \== floor,

Pi \== B.

can(move(B, floor, floor), [clear(B), on(B, floor)]) :-

block(B, _).

can(move(B1, Pi, B2), [clear(B1), clear(B2), on(B1, Pi)]) :-

block(B1, _),

block(B2, _),

block(Pi, _),

B1 \== B2,

B1 \== Pi,

Pi \== B2,

stable(B1, B2).

can(move(B1, floor, B2), [clear(B1), clear(B2), on(B1, floor)]) :-

block(B1, _),

block(B2, _),

B1 \== B2,

stable(B1, B2).

adds(move(B, Pi, Pj), [on(B, Pj), clear(Pi)]) :-

object(Pi),

object(Pj).

deletes(move(B, Pi, Pj), [on(B, Pi), clear(Pj)]) :-

object(Pi),

object(Pj).

5.1.5 IMPOSSIBILIDADES

impossible(on(X, X), _).

impossible(on(X, Y), Goals) :-

member(on(X, Y1), Goals), Y1 \== Y

member(on(X1, Y), Goals), X1 \== X.

5.1.6 PLANEJADORES

plan(State, Goals, Plan) :-

 between(0, 6, MaxDepth),

 plan(State, Goals, [], Plan, MaxDepth).

plan(State, Goals, _, [], _) :-

 satisfied(State, Goals).

plan(State, Goals, Visited, [Action|Rest], MaxDepth) :-

 MaxDepth > 0,

 NextDepth is MaxDepth - 1,

 seleciona(State, Goals, Goal),

 achieves(Action, Goal),

 can(Action, _),

 preserves(Action, Goals),

 \+ member(Action, Visited),

 regress(Goals, Action, RegressedGoals),

 plan(State, RegressedGoals, [Action|Visited], Rest, NextDepth).

satisfied(State, Goals) :-

 delete_all(Goals, State, []).

seleciona(_, Goals, Goal) :-

 member(Goal, Goals).

achieves(Action, Goal) :-

 adds(Action, Goals),

 member(Goal, Goals).

preserves(Action, Goals) :-

 deletes(Action, Relations),

 \+ (member(Goal, Relations),

 member(Goal, Goals)).

regress(Goals, Action, RegressedGoals) :-

 adds(Action, NewRelations),

 delete_all(Goals, NewRelations, RestGoals),

 can(Action, Condition),

 addnew(Condition, RestGoals, RegressedGoals).

addnew([], L, L).

addnew([Goal|_], Goals, _) :-

```

    impossible(Goal, Goals), !,
    fail.
addnew([X|L1], L2, L3) :-
    member(X, L2), !,
    addnew(L1, L2, L3).
addnew([X|L1], L2, [X|L3]) :-
    addnew(L1, L2, L3).
delete_all([], _, []).
delete_all([X|L1], L2, Diff) :-
    member(X, L2), !,
    delete_all(L1, L2, Diff).
delete_all([X|L1], L2, [X|Diff]) :-
    delete_all(L1, L2, Diff).

% Planejador em duas fases
plan_two_phase(State, Goals1, Goals2, Plan) :-
    plan(State, Goals1, Plan1),
    apply_plan(State, Plan1, MidState),
    plan(MidState, Goals2, Plan2),
    append(Plan1, Plan2, Plan).

% Aplica um plano a um estado
apply_plan(State, [], State).
apply_plan(State, [move(B, Pi, Pj)|Rest], FinalState) :-
    adds(move(B, Pi, Pj), Add),
    deletes(move(B, Pi, Pj), Del),
    delete_all(State, Del, Temp),
    append(Add, Temp, NewState),
    apply_plan(NewState, Rest, FinalState).

```

5.1.7 ESTADOS

```

state_s0_sit1([
    on(a, floor), on(b, floor),
    on(c, floor), on(d, a),

```

```
clear(c), clear(d), clear(b)
]).
```

```
state_s0_sit2([
    on(a, c), on(b, c),
    on(c, floor), on(d, floor),
    clear(a), clear(b), clear(d)
]).
```

5.2 TESTES

5.2.1 SITUAÇÃO 1 – S0 ATÉ Sf1

% Fase 1: mover d para o chão

% Fase 2: colocar a em cima de d

% Fase 3: colocar b em cima de d

teste_sit1_sf1(Plan) :-

```
state_s0_sit1(State),
plan(State, [on(d, floor)], Plan1),
apply_plan(State, Plan1, MidState1),
plan(MidState1, [on(a, d)], Plan2),
apply_plan(MidState1, Plan2, MidState2),
plan(MidState2, [on(b, d)], Plan3),
append(Plan1, Plan2, Temp),
append(Temp, Plan3, Plan).
```

5.2.2 SITUAÇÃO 2 – S0 ATÉ S5

% Fase 1: tirar a e b de cima de c

% Fase 2: colocar c em cima de d

% Fase 3: colocar a em cima de c

% Fase 4: colocar b em cima de c

teste_sit2_s5(Plan) :-

```
state_s0_sit2(State),
plan(State, [on(a, floor)], Plan1),
apply_plan(State, Plan1, MidState1),
plan(MidState1, [on(b, floor)], Plan2),
```

```

apply_plan(MidState1, Plan2, MidState2),
plan(MidState2, [on(c, d)], Plan3),
apply_plan(MidState2, Plan3, MidState3),
plan(MidState3, [on(a, c)], Plan4),
apply_plan(MidState3, Plan4, MidState4),
plan(MidState4, [on(b, c)], Plan5),
append(Plan1, Plan2, Temp1),
append(Temp1, Plan3, Temp2),
append(Temp2, Plan4, Temp3),
append(Temp3, Plan5, Plan).

```

5.2.3 SITUAÇÃO 3 – S0 ATÉ S7

% Fase 1: mover d para o chão

% Fase 2: colocar a em cima de c

% Fase 3: colocar b em cima de c

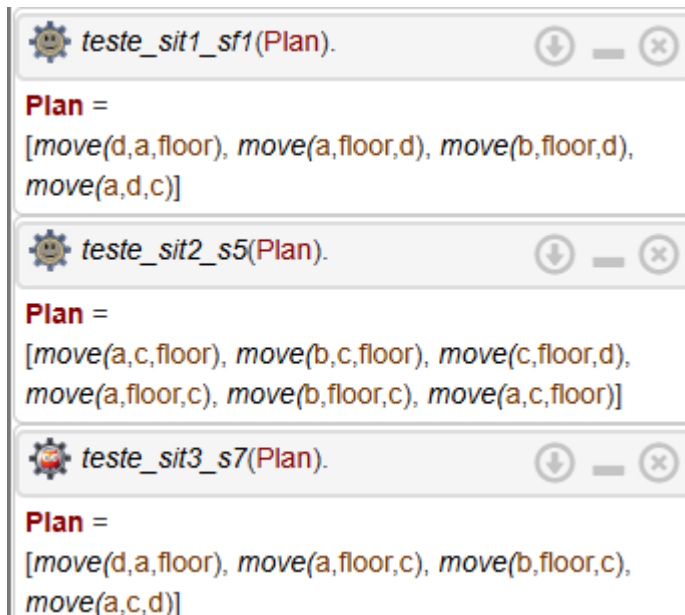
teste_sit3_s7(Plan) :-

```

state_s0_sit1(State),
plan(State, [on(d, floor)], Plan1),
apply_plan(State, Plan1, MidState1),
plan(MidState1, [on(a, c)], Plan2),
apply_plan(MidState1, Plan2, MidState2),
plan(MidState2, [on(b, c)], Plan3),
append(Plan1, Plan2, Temp),
append(Temp, Plan3, Plan).

```

5.3 RESULTADOS



Situação 1 — S0 até Sf1

Plano Manual:

1. move(d, a, floor)
2. move(a, floor, d)
3. move(b, floor, d)
4. move(c, floor, a)

Plano Gerado pelo Código:

1. move(d, a, floor)
2. move(a, floor, d)
3. move(b, floor, d)
4. move(a, d, c)

Análise: Os três primeiros passos são idênticos. A diferença está no quarto passo — o aluno colocou c em cima de a, enquanto o planejador colocou a em cima de c. Isso acontece porque o planejador por goal regression trabalha de trás para frente e encontrou uma solução ligeiramente diferente, mas igualmente válida para chegar ao objetivo.

Situação 2 — S0 até S5

Plano Manual:

1. move(a, c, floor)

2. move(b, c, floor)
3. move(c, floor, d)
4. move(a, floor, c)
5. move(b, floor, c)

Plano Gerado pelo Código:

1. move(a, c, floor)
2. move(b, c, floor)
3. move(c, floor, d)
4. move(a, floor, c)
5. move(b, floor, c)
6. move(a, c, floor)

Análise: Os cinco primeiros passos são idênticos ao plano manual. O planejador gerou um sexto passo extra move(a, c, floor) que é redundante — o objetivo já estava satisfeito após o quinto passo. Isso é uma característica conhecida do goal regression planning, onde o planejador pode gerar passos desnecessários quando os objetivos não estão perfeitamente ordenados.

Situação 3 — S0 até S7

Plano Manual:

1. move(d, a, floor)
2. move(a, floor, c)
3. move(b, floor, c)

Plano Gerado pelo Código:

1. move(d, a, floor)
2. move(a, floor, c)
3. move(b, floor, c)
4. move(a, c, d)

Análise: Os três primeiros passos são idênticos ao plano manual. O planejador gerou um quarto passo extra move(a, c, d) que move a de cima de c para cima de d. Isso acontece porque o planejador tentou satisfazer um objetivo adicional que não era necessário para chegar ao estado S7.

5.4 CONCLUSÃO GERAL

Situação	Passos Manual	Passos Código	Passos extras
Sit 1 $S_0 \rightarrow S_{f1}$	4	4	0
Sit 2 $S_0 \rightarrow S_5$	5	6	1
Sit 3 $S_0 \rightarrow S_7$	3	4	1

A principal diferença entre os planos manuais e os planos gerados pelo código é que o planejador por goal regression às vezes gera passos extras desnecessários. Isso acontece porque o algoritmo trabalha de trás para frente do objetivo para o estado inicial e pode incluir ações que satisfazem subobjetivos intermediários mas que não são necessárias para o objetivo final.

Os planos manuais são mais enxutos porque a equipe pensou de frente para trás, identificando diretamente a sequência mínima de ações necessárias. Já o planejador automático garante que todos os objetivos sejam satisfeitos, mas sem otimizar o número de passos.